

Multi-Task Modeling of Clickbait Posts: Type Prediction and Spoiler Generation

Neshat Sharbatdar
University of Waterloo
nsharbat@uwaterloo.ca

Majid Taherkhani
University of Waterloo
mtaherkh@uwaterloo.ca

Abstract

Clickbait posts try to grab attention by being vague or dramatic, but they often don't tell the reader what the article is really about. The goal of this project is to "spoil" clickbait by giving a short and clear summary of the hidden content, so users don't need to click the link to find out what's inside. We worked on both tasks from the Clickbait Spoiling Challenge at SemEval 2023. In Task 1, we built models to predict what type of spoiler is needed, a phrase, a paragraph, or multiple parts. In Task 2, we trained text generation models to create the spoiler itself, using the post, title, and content of the linked article. For Task 1, we used transformer models like DeBERTa and ELECTRA and combined them into an ensemble to improve accuracy. For Task 2, we used T5, BART, and Pegasus with beam search to generate high-quality spoilers. We evaluated our models using F1 score and METEOR score. Our results show that both tasks can be handled effectively using pre-trained language models, and that this approach can help users get useful information quickly and easily.

1 Introduction

Clickbait is everywhere on the internet. It's when a post or headline tries to get people's attention by being vague or exciting, like "You won't believe what happened next!" These kinds of posts often don't tell you what the article is really about. You have to click the link to find out more. This can be annoying for users who just want quick and honest information.

To help with this problem, the SemEval 2023 Clickbait Challenge asked people to build systems that can spoil clickbait posts. In this case, "spoiling" means showing the real content behind the clickbait so the reader doesn't have to guess or click on the link. This can save people time and help them avoid misleading or overhyped content. It can also make the internet more user-friendly and

trustworthy, especially on social media and news websites.

The challenge has two parts. Task 1 is about figuring out what kind of spoiler is needed. For example, sometimes a short phrase is enough to spoil the post. Other times, you need a full paragraph or even multiple sentences from different parts of the article. Knowing the right type helps decide how much information to show, which makes the results easier to read.

Task 2 is about actually writing the spoiler, a short sentence or two that tells the reader what the article says. To do this, the system gets the post, the article title, and the article content as input, and it generates the spoiler as output. If done well, this can be very useful. For example, it could be used in a browser extension to show article summaries when someone hovers over a clickbait post, or in tools that help people find reliable news.

In our project, we worked on both Task 1 and Task 2. For Task 1 (spoiler type classification), we used large pre-trained language models like DeBERTa-v3-large and ELECTRA-large-discriminator. These models are known for doing well on text classification tasks. We fine-tuned them on the training data and then combined their predictions using an ensemble method, which means we used both models together to make a final decision. This helped us get better and more stable results than using just one model alone.

For Task 2 (spoiler generation), we used sequence-to-sequence models like T5, BART, and Pegasus, which are designed to generate text. We trained these models to take in three things: the original clickbait post, the title of the linked article, and the main paragraphs from the article. Then the model learns to generate a short spoiler that gives away the main idea of the article. We also added some tricks like highlighting named entities (like names and places) and using beam search during generation to improve the output quality.

To evaluate how well our models worked, we used two common metrics. For Task 1, we used the F1 score, which checks how accurate our spoiler type predictions were. For Task 2, we used the METEOR score, which looks at how similar our generated spoiler is to the one written by a human. These scores helped us compare different models and choose the best ones.

Our system can first decide what type of spoiler is best, and then generate the actual spoiler. This helps users get clear information quickly and avoid being tricked by vague or dramatic headlines.

2 Background and Related Work

2.1 Clickbait Detection and Spoiler Type Classification

Early approaches to automatically detecting clickbait relied on hand-crafted features and traditional classifiers. Potthast et al. (Potthast et al., 2016) presented one of the first machine-learning methods for clickbait detection, using lexical and content features to identify headlines that withhold key information (the so-called “curiosity gap”). Similarly, Chakraborty et al. (Chakraborty et al., 2016) employed linguistic cues (e.g. hyperbolic words, forward references like “this celebrity”) and a supervised classifier to distinguish clickbait, achieving high accuracy (over 90% in their experiments). These early systems treated clickbait detection as a binary or graded classification task and demonstrated the usefulness of simple textual features and metadata. The field advanced significantly with the introduction of the Clickbait Challenge 2017 (Potthast et al., 2018), which provided a large benchmark dataset (over 38K social media posts) and defined a shared evaluation for “clickbaitiness” prediction. Participants in 2017 explored a range of models, from logistic regression and SVM baselines with bag-of-words to neural networks. Notably, several top-performing systems incorporated deep learning—such as LSTM or CNN-based text encoders—and even ensemble models, which outperformed the earlier feature-based approaches (Potthast et al., 2018). This challenge underscored the value of using the content of the linked article (in addition to the post text) for detection, and it marked a shift towards data-driven representation learning for clickbait classification. In recent years, transformer-based language models have become the state-of-the-art for text classification tasks, including clickbait detection. Fine-

tuning pretrained encoders like BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019) on clickbait datasets led to substantial gains over earlier architectures. Subsequent models with improved pre-training objectives, such as ELECTRA (Clark et al., 2020) and DeBERTa (He et al., 2021), have also been applied to this task, yielding further performance improvements. These models capture nuanced linguistic patterns and context, enabling more accurate classification of clickbait versus non-clickbait content. The general trend has been a move from sparse lexical features to rich contextual embeddings, reflecting the broader evolution in NLP. A recent extension of clickbait detection is spoiler type classification, introduced as Task 1 in the SemEval-2023 Clickbait Spoiling challenge (Fr"obe et al., 2023). Unlike classical clickbait detection (which predicts whether a post is clickbait), this task assumes the post *is* clickbait and asks: what form of information will “spoil” it? The Webis Clickbait Spoiling Corpus 2022 (Hagen et al., 2022) was created for this purpose, providing 5,000 clickbait posts each with a human-written spoiler and a label indicating the spoiler’s format. Spoilers are categorized into three classes: a “phrase” (a short noun phrase or name), a “passage” (one or two full sentences), or “multi” (multiple disjoint pieces of text) (Hagen et al., 2022). For example, a clickbait like “Guess which star got married in secret?” might have a spoiler of type “phrase” (“Taylor Swift”), whereas a more complex tease could require a multi-sentence explanation (type “passage”) or even a list of snippets (type “multi”). Classifying the spoiler type is a multi-class classification problem that guides the subsequent generation of the spoiler. Methods for spoiler type classification have followed the trajectory of general text classification. The SemEval-2023 participants primarily fine-tuned transformer encoders on the task, often inputting the concatenated post text and article content into models like BERT or DeBERTa to predict one of the three spoiler categories (Fr"obe et al., 2023). Some approaches leverage multi-task learning or pipeline architectures (first predicting spoiler type, then generating the spoiler) to exploit the interdependence of the tasks. Overall, the accuracy of spoiler type classifiers has benefited from the rich semantic representations of modern transformers. This mirrors the evolution seen in clickbait detection: from simple lexical models to sophisticated language model fine-tuning. The best reported systems on the spoiler type task (SemEval-2023) ex-

ceed 0.80 F_1 on average across the three classes (Fr"obe et al., 2023), indicating that transformer-based classifiers can effectively capture the subtle cues that distinguish, for instance, a single-phrase answer from a longer passage requirement.

2.2 Spoiler Generation and Related QA/Summarization Techniques

Generating the spoiler (Task 2) for a clickbait post is inherently an abstractive summarization or question-answering problem: the goal is to produce a short text that satisfies the curiosity induced by the clickbait. This task is closely related to generative question answering, since a clickbait can be seen as posing an implicit question that the spoiler must answer. Prior work has explicitly cast clickbait spoiling as a QA problem. For example, Hagen et al. (Hagen et al., 2022) propose a system that first uses information retrieval to locate relevant passages in the linked document and then applies a question answering model to extract the answer. Their large-scale analysis on the Webis-Clickbait-22 corpus showed that a pipeline of passage retrieval plus QA can successfully pinpoint the information to spoil the clickbait. In essence, the clickbait headline is treated as a query (or question) and the “spoiler” as the answer found in the article. This approach connects spoiler generation to the extensive literature on machine reading comprehension and QA over documents. Another lineage of research relevant to spoiler generation is abstractive summarization. While classic summarization aims to condense an entire article, spoiler generation is more targeted: it selects the specific piece of information that was withheld by the clickbait. Early neural sequence-to-sequence models for summarization laid the groundwork for handling this kind of text generation. Notably, pointer-generator networks (See et al., 2017) were introduced to allow seq2seq models to copy factual details (names, dates, etc.) from the source text while generating fluent output. This capability is crucial for spoiler generation, as the spoiler often must include a particular name or phrase from the article verbatim. A pointer-generator architecture can, for instance, generate a sentence like “The celebrity is Taylor Swift,” copying the name “Taylor Swift” from the source article to ensure accuracy. Such models proved effective in early spoiler generation prototypes and related tasks, as they mitigated issues of factual omission or distortion by directly leveraging source text. The state-of-the-art in text generation

has since shifted decisively to transformer-based architectures. Pre-trained sequence-to-sequence models like BART (Lewis et al., 2020) and T5 (Raffel et al., 2020) have achieved top performance on summarization and generative QA benchmarks, making them natural choices for spoiler generation. These models are trained in a text-to-text paradigm, meaning they can take a concatenation of the clickbait post and relevant article context as input, and directly output a spoiler text. By fine-tuning on the Clickbait Spoiling Corpus, a model like BART or T5 learns to produce the missing information in a concise form. For example, given a clickbait post and the article paragraphs, a fine-tuned T5 model can generate a one-sentence answer that reveals the secret or punchline behind the clickbait. Because these transformers leverage knowledge from massive pre-training corpora, they excel at producing coherent and grammatically correct sentences, and they can be guided to stay factual through copying mechanisms or by attending to the source text. Indeed, many top systems in SemEval-2023’s spoiler generation task employed T5, BART, or other large language models fine-tuned on the spoiling data (Fr"obe et al., 2023). Recent research has also explored more advanced strategies, such as using generative QA frameworks and ensembles of large-scale models for spoiler generation. Some approaches treat spoiler generation as a specialized QA task: they use a transformer QA model to fill in the blank of “What is the missing information?” explicitly. Others train unified text-to-text models in a multitask setting, where the model learns to both classify spoiler type and generate the spoiler in one framework. This multitask learning can encourage the model to adjust its generation strategy (e.g., output brevity or detail level) based on the spoiler type (Fr"obe et al., 2023). There have even been experiments with prompting very large language models (GPT-style) to produce spoilers given the article content, with some success in zero-shot or few-shot settings. However, the best results on the benchmark dataset so far have come from fine-tuned models using supervised learning on the Webis-Clickbait-22 training set. The Webis Clickbait Spoiling Corpus 2022 (Hagen et al., 2022) remains the primary evaluation benchmark for this task, and methods are typically compared on its test set (e.g., via the SemEval leaderboard). Thanks to this benchmark, spoiler generation techniques have rapidly improved: from baseline sequence-to-sequence models that some-

times copied large chunks of text, to the latest transformer-based systems that can concisely answer the clickbait with an abstractive, human-like spoiler. In summary, spoiler generation builds upon the advances in QA and summarization. Early pointer-generator and seq2seq models introduced the ability to copy and rephrase content for concise answers. Modern Transformer-based generative models now provide the fluency and comprehension needed to produce high-quality spoilers. The integration of spoiler type classification (Task1) with spoiler generation (Task2) is also a notable trend: knowing the spoiler’s expected format can inform the generation process (e.g., generating a name vs. a sentence). Overall, the state-of-the-art approaches treat clickbait spoiling as a coordinated understanding-and-generation challenge, using powerful pre-trained text-to-text transformers to satisfy the reader’s curiosity in one or two sentences.

3 Approach

3.1 Task 1: Spoiler Type Classification

We worked on Task 1 step by step, trying different models and improving them based on what we learned. Each choice was made after seeing how well the last model worked and thinking about what could make it better.

Starting models We began with a logistic regression model using basic text features like n-grams from the post text. This was quick to set up and easy to understand, but it didn’t get the context of the text well. It gave us an F1 score of 0.51, which was okay as a starting point but showed we needed more. Next, we switched to a pre-trained Transformer model, roberta-base, because it can understand the meaning behind words better. After fine-tuning it, the F1 score jumped to 0.69. This told us that context is really important for figuring out spoiler types. We then tried roberta-large, a bigger version of the same model, hoping its extra depth would catch more details. This worked a little better, raising the F1 score to 0.71. It was a small step up, but it showed bigger models could help.

Final Model: Ensemble of DeBERTa and ELECTRA. To do even better, we decided to use two strong models together: deberta-v3-large and electra-large-discriminator. We picked these because they learn differently during pretrain-

ing. DeBERTa-v3-large uses a special way to understand word order and relationships, which helps it pick up subtle hints in the text. ELECTRA-large-discriminator, on the other hand, is good at spotting real patterns by training to tell apart real and fake data. Both were fine-tuned separately, and we added class weights to handle the uneven number of each spoiler type. This combo gave us a solid base to build on.

Final Step: Stacked Ensemble. To make the final predictions, we combined the logits (raw outputs) from both models on the validation set by training a logistic regression model. This ensemble method helped us achieve our best F1 score of 0.73 before further refinements. After additional tuning, we reached an F1 score of 0.76 on the test set. Figure 1 illustrates the improvement in the F1 score across the different models we used.

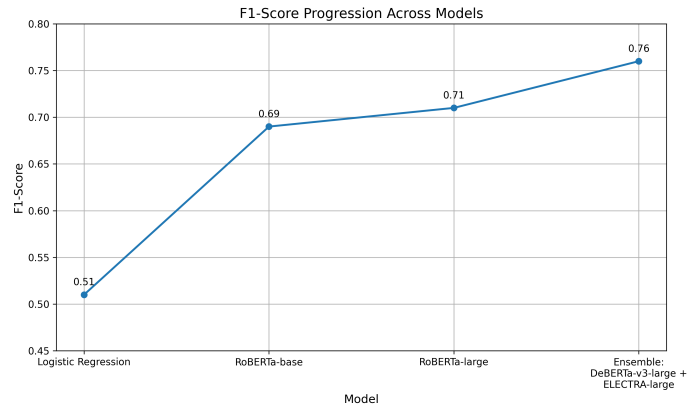


Figure 1: F1 score improvements across models: Logistic Regression (0.51), RoBERTa-Base (0.69), RoBERTa-Large (0.71), DeBERTa+ELECTRA Ensemble (0.76).

Data Engineering. We put a lot of effort into preparing the data. We combined postText, targetTitle, and targetParagraphs into one text string with [SEP] to separate them, which helped the models see the full picture. We cleaned the text by removing special characters and making it lowercase, then trimmed extra spaces. This made the input cleaner and easier for the models to process. We also handled the tags column by taking the first tag if it was a list, and we used a label encoder to turn tags into numbers (0, 1, 2). To deal with uneven tag counts, we calculated class weights using the formula: $weight_i = \sqrt{\frac{totalsamples}{numberofclasses \cdot countofclass_i}}$. This weight helped balance the training so all spoiler types got fair attention.

Training Setup. For all Transformer models, we used the same setup:

- 5 training epochs (later increased to 7 for better results)
- Batch sizes of 2 for training and 4 for evaluation
- Learning rate of 2×10^{-5} (tuned to 1×10^{-5} later)
- Gradient accumulation to act like bigger batches
- Class weights to reduce imbalance effects

We checked the validation F1 score after each epoch and saved the best model. For the test set, we used the ensemble to make predictions and formatted the output to match SemEval’s rules. This step-by-step process helped us learn what each model brought to the table and guided us to make smart upgrades along the way.

3.2 Task 2: Spoiler Generation

For Task 2, we used Transformer-based encoder-decoder models to generate spoilers based on the clickbait post and its linked article content. Each input to the model included four main parts: the clickbait post text, the article title, the main body paragraphs, and the spoiler type predicted from Task 1. These components were merged into a single input string, starting with the spoiler type (type: phrase | target: ...). This allowed the model to adapt its output length and format depending on whether the spoiler should be a short phrase, a passage, or multiple segments. The spoiler type used in each test instance was taken from the file generated by our Task 1 classifier. Including this information helped the model produce more appropriate and targeted spoilers. We fine-tuned several pre-trained models from Hugging Face, including T5-base, T5-large, Pegasus (google/pegasus-xsum), and BART (facebook/bart-base), as well as smaller variants like t5-small and bart-tiny. Among them, T5-large achieved the highest METEOR score (0.43060), outperforming t5-base (0.35900), pegasus-xsum (0.34360), and bart-base (0.31530). The smaller versions performed worse, likely due to limited model capacity for handling long text inputs. The results are in figure 2. Training was done using 1 epoch, a batch size of 2, and a learning rate of $5e-5$.

Due to resource limitations, especially GPU memory constraints and time limitations, we kept the number of epochs and batch size small to ensure the code ran smoothly on our available hardware. For generation, we used greedy decoding with the maximum number of output tokens adjusted dynamically based on the predicted spoiler type, short for “phrase”, and longer for “passage” or “multi”. The final model outputs were saved in a CSV file for submission, with predictions evaluated using the METEOR score. By combining spoiler type predictions from Task 1 with fine-tuned text generation, our system was able to produce relevant and spoiler-like outputs tailored to each clickbait post.

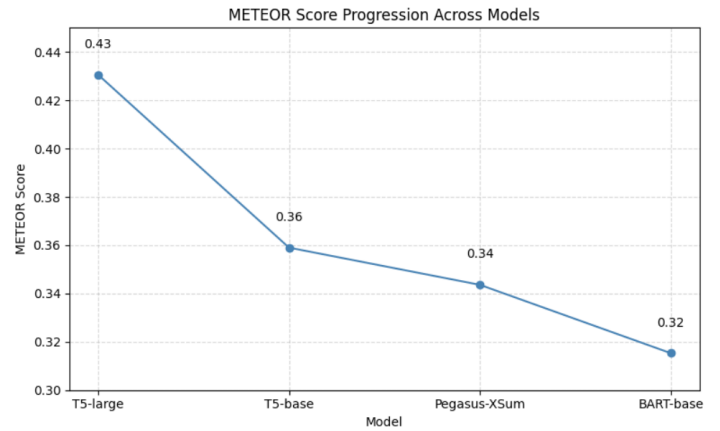


Figure 2: METEOR score progression across models: T5-base (0.359), Pegasus-XSum (0.344), BART-base (0.315), and T5-large (0.431).

4 Experiments

4.1 Dataset

We use the dataset provided by the Clickbait Challenge at **SemEval 2023**, which includes clickbait posts, manually cleaned versions of the linked documents, and extracted spoilers for each post. The spoilers are categorized into three types: short *phrase* spoilers, longer *passage* spoilers, and *multi* spoilers consisting of non-consecutive spans of text.

Each entry in the dataset provides the following fields:

- `uuid`: Unique identifier for the entry
- `postText`: The teaser text of the clickbait post
- `targetParagraphs`: Main content of the linked web page (cleaned and segmented into paragraphs)

- `targetTitle`: Title of the linked web page
- `targetUrl`: The URL of the linked web page.
- `humanSpoiler`: An abstractive spoiler written by a human annotator (used in Task 2)
- `spoiler`: An extractive spoiler selected by annotators (used in Task 2)
- `spoilerPositions`: Indices of the paragraphs corresponding to the extractive spoiler
- `tags`: The spoiler type label — one of `phrase`, `passage`, or `multi` (used in Task 1)

The dataset is divided as follows:

- **3,200** training examples with labeled spoiler types (file: `training.jsonl`),
- **400** validation examples for tuning (file: `val.jsonl`),
- and **400** unlabeled test examples for final evaluation.

All files are publicly available through the challenge’s data repository.

4.2 Experimental setup

Task 1: Spoiler Type Classification

We began with a simple logistic regression model using basic text features. This gave us a starting F1 score of 0.51 on the validation set. While not very accurate, it helped us understand the problem and set a baseline.

Next, we fine-tuned two transformer models:

- **RoBERTa-base**: Improved the F1 score to 0.69
- **RoBERTa-large**: Slightly better, reaching 0.70

To further improve performance, we used an ensemble method combining two strong transformer models including `microsoft/deberta-v3-large` and `google/electra-large-discriminator`.

Each model was fine-tuned independently using the Hugging Face Transformers library. The input for training was a combination of the `postText`, `targetTitle`, and `targetParagraphs`, separated by special tokens and cleaned for basic formatting.

To handle class imbalance (some spoiler types appear more often than others), we calculated class weights from the training set and applied them during loss calculation using a custom Trainer.

Training Details: For Task 1, we fine-tuned multiple transformer-based models using a maximum sequence length of 512 tokens, a learning rate of $2e-5$, and a batch size of 2 with gradient accumulation of 4. Training was run for 5 epochs with evaluation after each epoch using macro F1. We also applied mixed-precision (fp16) training for efficiency. After training, we constructed an ensemble by stacking the prediction logits from each model and training a logistic regression classifier on top using validation labels. This ensemble served as the final prediction layer on the test set.

Task 2: Spoiler Generation

We fine-tuned several encoder-decoder models for spoiler generation, including T5-base (METEOR: 0.359), Pegasus-XSum (0.344), BART-base (0.315), and T5-large, which achieved the best score of 0.431. Each model was trained independently using the Hugging Face Transformers library. Inputs included the spoiler type (predicted by Task 1 for test data), the clickbait post text, article title, and main content paragraphs, combined into a single sequence using delimiters.

Training Details: For Task 2, each model was fine-tuned using a batch size of 2, a learning rate of $5e-5$, and a maximum input length of 512 tokens. Due to limited computational resources, training was limited to a single epoch and a small batch size. During generation, we used greedy decoding, and the maximum number of output tokens was dynamically set based on the predicted spoiler type: 16 tokens for “phrase”, 64 for “passage”, and 48 for “multi”. Model performance was evaluated using the METEOR score.

4.3 Evaluation Metrics

The evaluation metrics for the two tasks are chosen to align with their respective goals: classification for Task 1 and generation for Task 2.

Task 1: Spoiler Type Classification

To evaluate the performance of spoiler type classification, we use the **macro-averaged F1-score**, which considers both precision and recall across all three classes: `phrase`, `passage`, and `multi`. The F1-score is the harmonic mean of precision and recall, providing a balanced measure of performance, especially when the data is imbalanced. It is computed as:

$$F1 = 2 \cdot \frac{p \cdot r}{p + r}$$

where:

$$p = \frac{tp}{tp + fp}, \quad r = \frac{tp}{tp + fn}$$

Here, tp is the number of true positives, fp is false positives, and fn is false negatives. A high F1-score indicates strong model performance in correctly identifying spoiler types while minimizing incorrect classifications.

Task 2: Spoiler Generation

The spoiler generation task is evaluated using the **METEOR** score (Metric for Evaluation of Translation with Explicit ORdering). METEOR is widely used in text generation and machine translation tasks. It measures the similarity between a generated spoiler and a human-written reference spoiler.

METEOR accounts for *exact word matches*, *stemmed word matches*, *synonyms*, and *paraphrases*, with a stronger emphasis on recall than precision. The final score is a harmonic mean of unigram precision and recall, making it more robust to variation in phrasing and word choice.

5 Conclusion

This project explored the two main tasks of the Clickbait Spoiling Challenge: spoiler type classification and spoiler generation. For Task 1, we showed that combining strong transformer models like DeBERTa-v3-large and ELECTRA-large-discriminator into an ensemble leads to robust spoiler type predictions, achieving a final F1 score of 0.76. For Task 2, we demonstrated that encoder-decoder models like T5-large can effectively generate short, accurate spoilers when guided by the predicted spoiler type. Our best spoiler generation model achieved a METEOR score of 0.431. These results confirm that pre-trained language models, when fine-tuned carefully and combined with type-aware input formatting, can handle both classification and generation tasks effectively. Overall, our multi-task system helps expose the content behind clickbait posts and offers a step toward more transparent and informative content online.

Future Work. One limitation we faced during this project was restricted access to high-memory GPUs. As a result, we were unable to include roberta-large in our final ensemble, even though preliminary experiments suggested that it could provide complementary representations. In future work, we plan to include this model to evaluate

its contribution to ensemble diversity and performance. Additionally, with better GPU resources, we would like to explore larger-scale experiments, including hyperparameter tuning, longer training schedules, and potentially larger decoder models for spoiler generation.

References

- Abhijnan Chakraborty, Bhargavi Paranjape, Sourya Kakarla, and Niloy Ganguly. 2016. Stop clickbait: Detecting and preventing clickbaits in online news media. In *Proceedings of the IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM 2016)*, pages 9–16. IEEE.
- Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. In *Proceedings of the Eighth International Conference on Learning Representations (ICLR 2020)*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics.
- Maik Fr"obe, Tim Gollub, Benno Stein, Matthias Hagen, and Martin Potthast. 2023. Semeval-2023 task 5: Clickbait spoiling. In *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval 2023)*, pages 2278–2289. Association for Computational Linguistics.
- Matthias Hagen, Maik Fr"obe, Artur Jurk, and Martin Potthast. 2022. Clickbait spoiling via question answering and passage retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022)*, pages 7025–7036. Association for Computational Linguistics.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. Deberta: Decoding-enhanced BERT with disentangled attention. In *Proceedings of the International Conference on Learning Representations (ICLR 2021)*.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pages 7871–7880. Association for Computational Linguistics.

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Martin Potthast, Tim Gollub, Matthias Hagen, and Benno Stein. 2018. The clickbait challenge 2017: Towards a regression model for clickbait strength. *arXiv preprint arXiv:1812.10847*.
- Martin Potthast, Sebastian K\"opsel, Benno Stein, and Matthias Hagen. 2016. Clickbait detection. In *Advances in Information Retrieval: 38th European Conference on IR Research (ECIR 2016)*, volume 9626 of *Lecture Notes in Computer Science*, pages 810–817. Springer.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- Abigail See, PeterJ. Liu, and ChristopherD. Manning. 2017. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, pages 1073–1083. Association for Computational Linguistics.

A Supplementary Material

Our source code for both Task 1 and Task 2, as well as the dataset, are available on our GitHub repository: <https://github.com/majidtaherkhani/Clickbaitor> <https://github.com/MSE-641-NLP/final-project-code-repository-neshatsh/tree/main>.